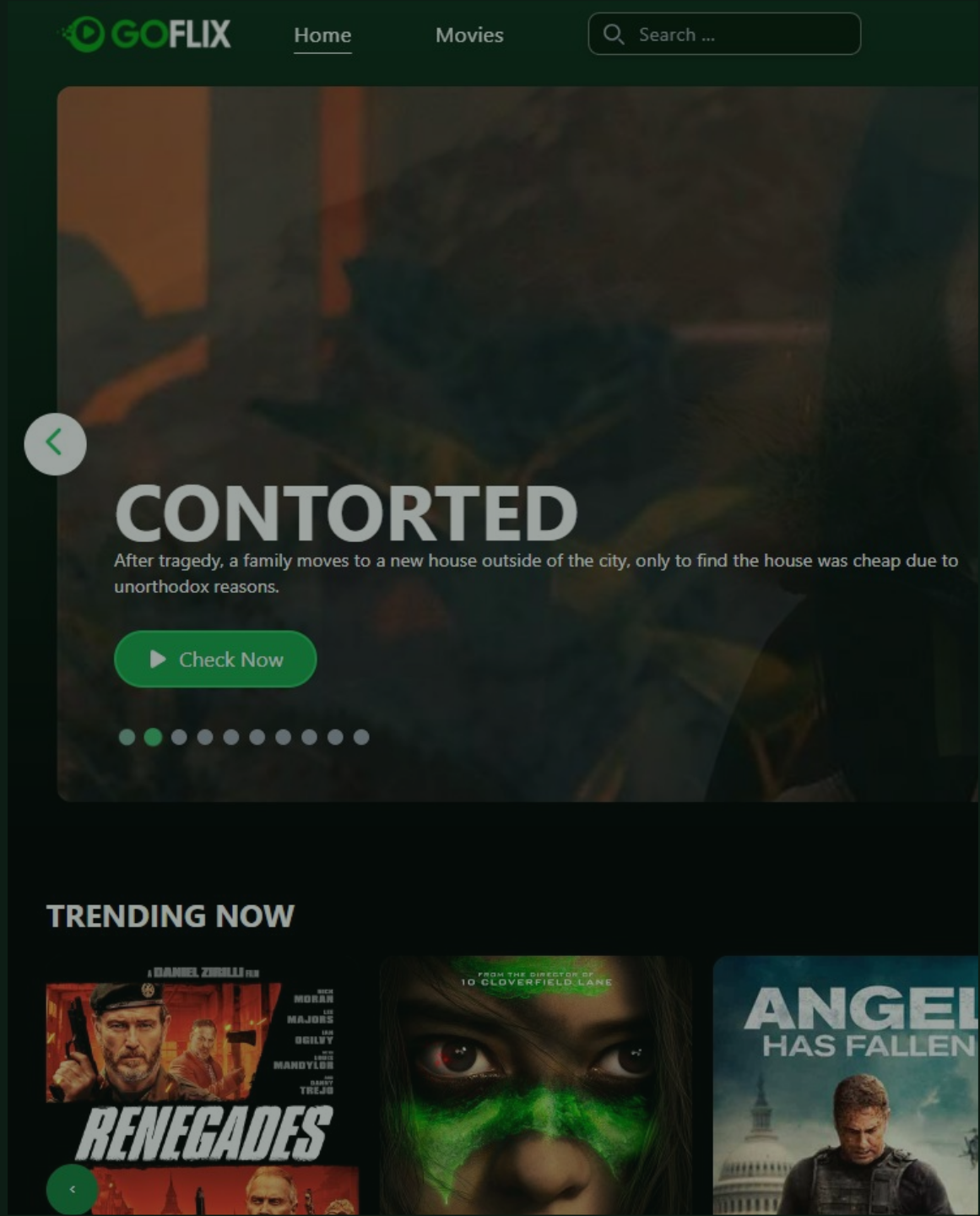
STREAMING PLATFORM

Goflix

The admin dashboard behind the catalog, the plans, and who may watch what

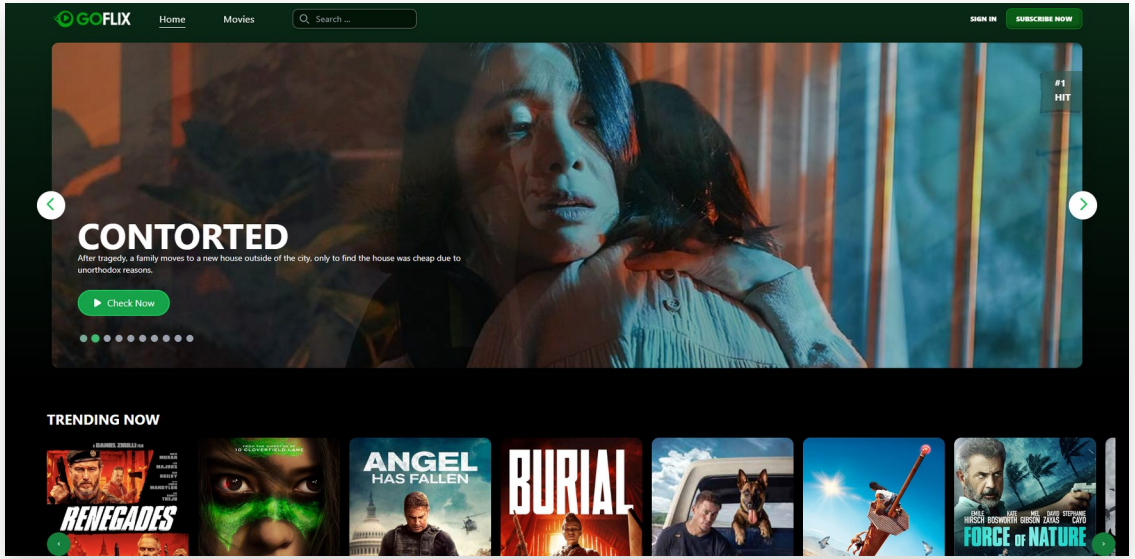
Enrico Saputra · Backend developer, Laravel dashboard

Client code and data are private. The demo rebuilds the Laravel dashboard on the real data model, with made-up data.

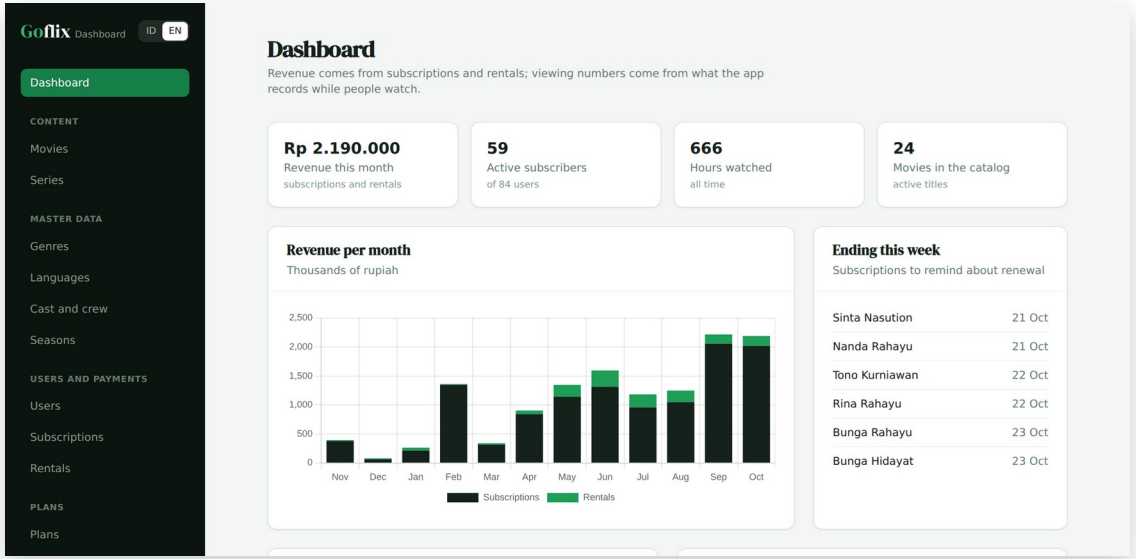


Viewers see this; the team works here

01



Viewer app · Vue + Go



Dashboard · Laravel · my part

The content team adds movies and episodes, the business team sets plans and coupons, and support answers "why can't I watch this?"

Three ways to watch

02



Free

Free titles play for anyone with an active account.



Plan

A monthly or yearly plan opens every premium title.



Rental

One premium movie, for a few days, without a plan.

My part: the catalog, plans and coupons, users and payments, and the reports on them.

About the app

03

Goflix is a streaming platform for local and international movies and series, on the web (Vue.js) and on mobile. Some titles are free; premium titles open with a monthly or yearly plan, or by renting a single movie.

Who uses it

Viewers

Watch, save titles and continue where they stopped.

Content and business team

Manage the catalog, plans, prices and coupons in the dashboard.

Support team

Check why a user can or cannot watch something.

User journey

1. A viewer signs in on the web or mobile app.
2. They browse movies and series by genre.
3. Free titles play right away; premium titles ask for a plan or a rental.
4. They pay for a plan or rental through the payment gateway and get access.
5. Viewing history is saved, so they can continue from where they stopped.

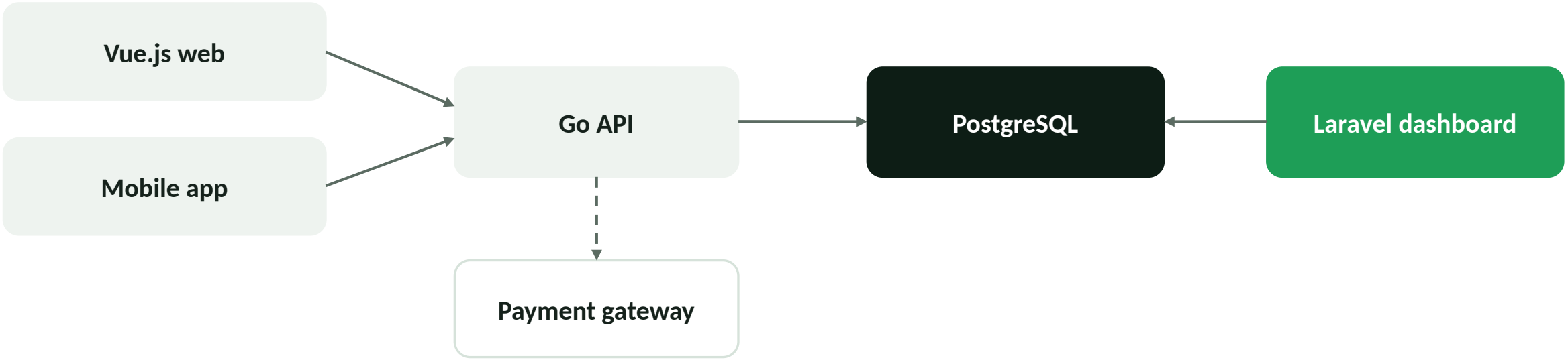
What the dashboard contains

04

Menu	What it does	Data
Dashboard	Revenue per month, active subscribers, hours watched, top movies and genres, plans sold, sign-in methods	subscriptions, rentals, watch_history
Movies, Series	The catalog: movies, series with episodes per season, free or premium, rent price	movies, shows, episodes
Genres, Languages, Cast, Seasons	Master data for the catalog	genres, languages, people, seasons
Users	Subscription status, payments, continue watching and an access check per title	users, watch_history
Subscriptions, Rentals	Payment records; can be added, never edited	subscriptions, rentals
Plans, Coupons	Plan price and length, plan-card lines, discount coupons	packages, coupons

Three parts, one database

05



Two writers

The API writes activity and in-app payments; the dashboard writes the catalog, plans and corrections.

The server decides

The app never decides access on its own. The dashboard shows support the same rule.

No API in the dashboard

Everything the apps need goes through the Go API.

The data model in three groups

06

Catalog

- movies
- shows
- episodes
- seasons
- genres
- languages
- people

Users and money

- users
- packages
- package_features
- subscriptions
- rentals
- coupons

Activity

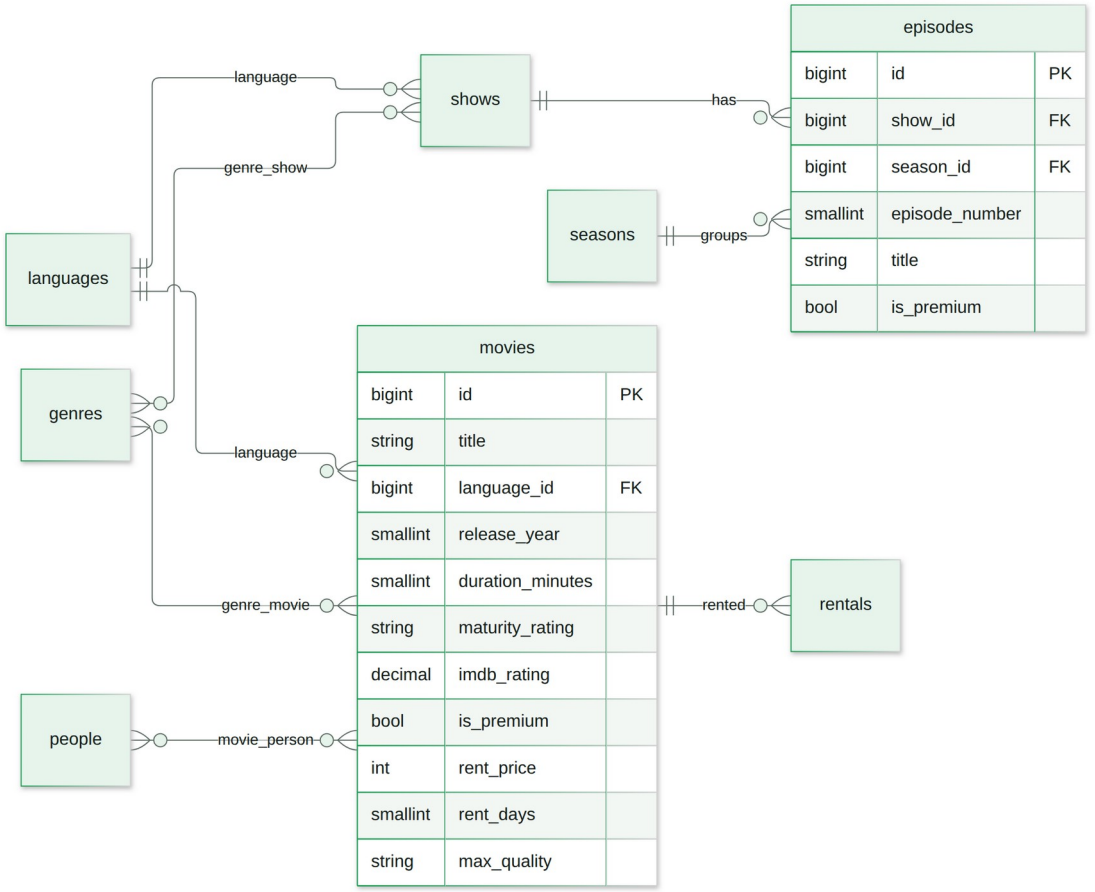
- watch_history
- bookmarks

Written by the apps while people watch. The dashboard only reads it, for reports and continue watching.

users.subscription_ends_on is the source of truth for a fast access check; only SubscriptionService may change it.

Database 1/2: the catalog

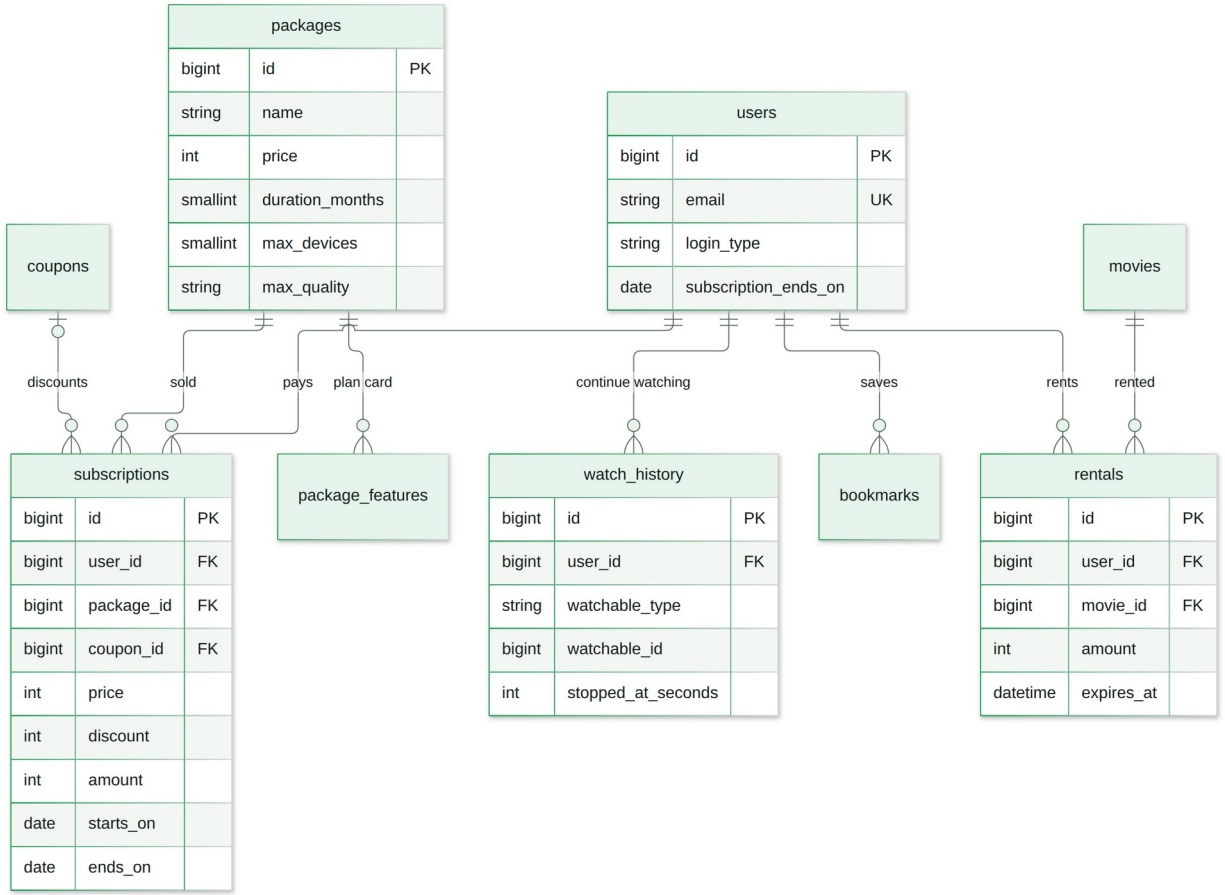
07



Movies, series, episodes and seasons, with pivot tables for genres, languages and cast. Boxes without columns are tables from the other part.

Database 2/2: users and payments

08



Plans, subscriptions, rentals and coupons. users.subscription_ends_on serves the fast access check. Boxes without columns are tables from the other part.

Relations: from text to pivot tables

09

Production

```
video
  id: 12
  category_id: '6,12,19'
  cast_id: '1,108,109,110'
```

Cannot be joined or indexed; deleted ids stay behind.

Demo

```
genre_movie
  (movie_id 12, genre_id 6)
  (movie_id 12, genre_id 12)
  (movie_id 12, genre_id 19)
```

Foreign keys, normal joins, and the genre report is one query.

Gradual migration: add the pivot, fill it from the old column, move readers one by one, then drop the old column.

Renewing without losing paid days

10

Active until 20 Mar, buys 12 months on 10 Mar

old: until 20 Mar	new: 21 Mar to 20 Mar next year

Ended 31 Jan, buys 1 month on 10 Mar

lapsed	10 Mar to 9 Apr

One place

Only SubscriptionService moves users.subscription_ends_on.

One transaction

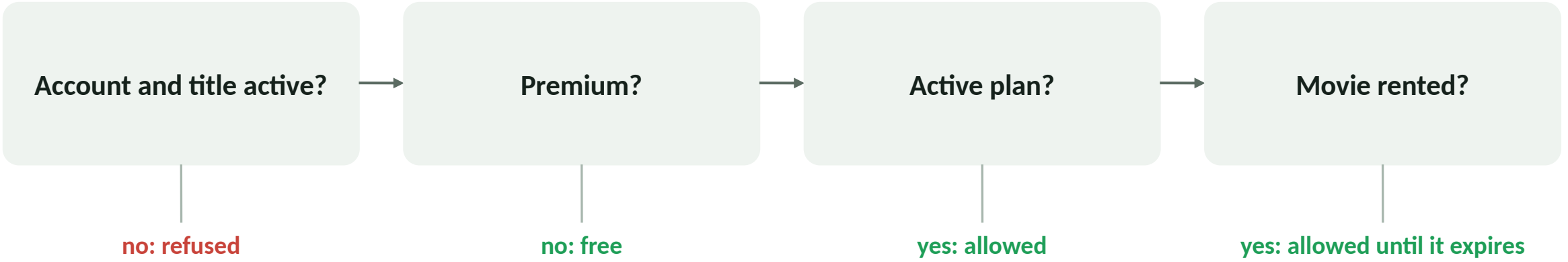
The payment and the end date are saved together, or neither is.

A row lock

lockForUpdate makes a second payment wait and count from the new date.

Who may watch what

11



Refused: needs a plan, or a rental if the movie has a rent price

An episode is premium on its own or through its series. One AccessService is used everywhere, and the dashboard shows it per user so support can answer complaints.

Rentals and coupons

12

Renting one movie

- Only premium movies with a rent price
- Lasts rent_days days from payment
- Active subscribers cannot rent: it is already included
- The same movie cannot be rented twice at once

Coupons

- Fixed amount or percentage
- Only valid between its start and end dates
- Never takes a price below zero
- A single-use coupon works once per user

Example: an active subscriber tries to rent a premium movie → refused, because the plan already includes it. Nobody pays twice, and refund requests drop.

Why this stack

13

Choice	Why
Laravel 10	The dashboard: forms, validation, CRUD. My part
PostgreSQL	Shared with the Go API; transactions and row locks
Go	The apps' API: heavy playback traffic
Blade + Chart.js	An internal admin needs no SPA; one chart needs only Chart.js

A renewal is one transaction


```
return DB::transaction(function () use (...) {
    $user = User::lockForUpdate()
        ->findOrFail($user->id);
    $startsOn = $user->subscription_ends_on
        ?->gte($today)
        ? $user->subscription_ends_on->addDay()
        : $today;
    $endsOn = $startsOn->copy()
        ->addMonthsNoOverflow($months)->subDay();
    $user->subscriptions()->create([...]);
    $user->update(['subscription_ends_on' => $endsOn]);
});
```

Rules in services

Thin controllers; plans, rentals, coupons and access are plain classes.

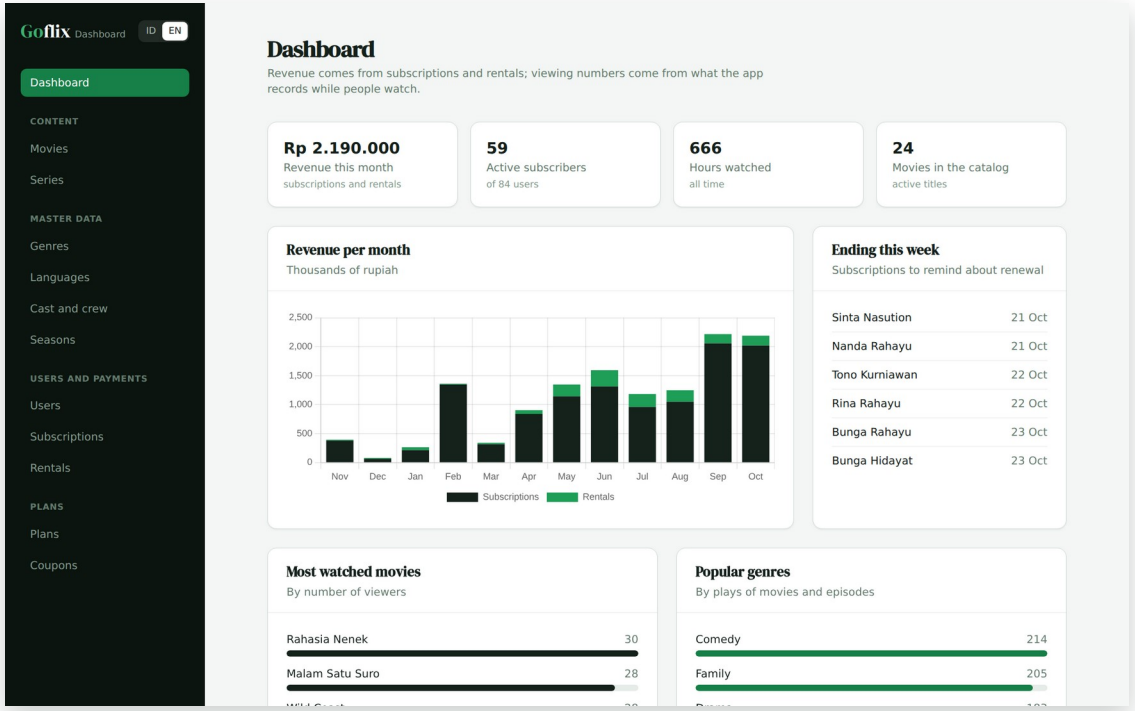
Enums, not numbers

Sign-in, role and coupon type are typed.

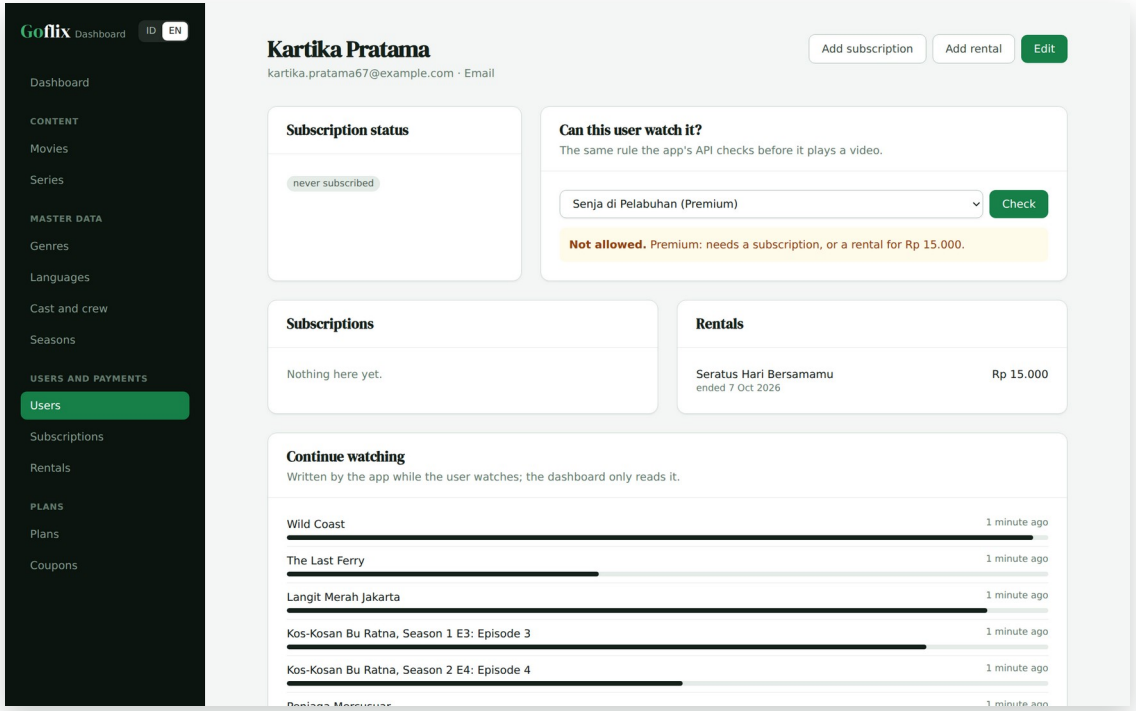
Tests on the rules

33 feature tests.

Demo: revenue, then a user's access check



Dashboard: plan and rental revenue per month



User page: status, payments and the access check

Demo and slides

Live demo	https://goflix-dashboard-demo.vercel.app
Slides (PDF)	https://goflix-dashboard-demo.vercel.app/goflix-walkthrough.pdf
Slides (PPTX)	https://goflix-dashboard-demo.vercel.app/goflix-walkthrough.pptx

The demo code is in a private GitHub repo; access is given during the interview. The dashboard speaks Bahasa Indonesia and English; demo data is restored every 3 hours.